PROJECT REPORT

Signature Verification System for Banks

Course: DIGITAL SIGNAL PROCESSING (ECN15101)

Submitted by

Ashish Singh • 20244045

Ayush Agarwal • 20244047

Ayush Jadaun • 20244048

Ayush Kaushal • 20244049

Submitted to: **Dr. Rajeev Gupta**



Department of Electronics and Communication Engineering

Motilal Nehru National Institute of Technology Allahabad

Prayagraj – 211004, Uttar Pradesh, India

Academic Session 2026–2027

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to everyone who helped us in completing this project on a signature verification system for banks, carried out as part of the course Digital Signal Processing (ECN15101).

We are deeply thankful to our project guide and to the faculty members of the department for their constant encouragement, valuable suggestions and patient review of our work at every stage. Their teaching of digital signal processing gave us the foundation on which every module of this system is built. We would especially like to thank:

1. **Dr. Rajeev Gupta**

   Department of Electronics and Communication Engineering, MNNIT Allahabad

2. Prof. _____

   (Designation, Department)

3. Prof. _____

   (Designation, Department)

We also thank the Head of the Department of Electronics and Communication Engineering, Motilal Nehru National Institute of Technology Allahabad, and the laboratory staff for providing access to the computing facilities and MATLAB licences used throughout the project. We acknowledge the Center of Excellence for Document Analysis and Recognition (CEDAR), University at Buffalo, for making their signature database publicly available for research.

Finally, we thank our families and friends for their support and understanding during the course of this work.

PREFACE

Handwritten signatures remain the most widely accepted means of authorising a financial transaction. Every cheque, withdrawal slip and account opening form in a bank carries one, and the burden of checking it usually falls on a clerk comparing the document against a specimen card by eye. This is slow, subjective and vulnerable to skilled forgeries.

This report documents the design and implementation of an offline signature verification system written entirely in MATLAB. The central idea is to treat a scanned signature as a signal: the image is cleaned with classical filters, reduced to one-dimensional projection signals, analysed in the frequency domain with the Fast Fourier Transform, decomposed with a Haar wavelet transform and compared against the stored specimens with two-dimensional normalised cross-correlation. A small writer-independent machine learning classifier is layered on top of the same DSP features, and both scores are fused into a single decision.

The report is organised in the order in which a signature travels through the system. After the introduction and theoretical background, separate chapters cover preprocessing, feature extraction, matching and scoring, threshold calibration and parameter tuning, the graphical user interface, and the experimental evaluation on the public CEDAR database. Relevant MATLAB code is reproduced directly from the project source so that each explanation can be checked against the implementation. Applications, future scope and conclusions follow, and the appendices give the complete file listing, reproduction steps and sample log output.

We hope the report serves both as a record of our work and as a practical example of how core DSP concepts taught in the classroom can be applied to a real problem in banking security.

INDEX

ACKNOWLEDGEMENT i
PREFACE ii
INDEX iii
LIST OF FIGURES vi
LIST OF TABLES vi
1. INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
2. THEORETICAL BACKGROUND 3
2.1 Images as Two-Dimensional Signals 3
2.2 Median Filtering 3
2.3 Otsu Thresholding 3
2.4 Discrete Fourier Transform 3
2.5 Discrete Wavelet Transform (Haar) 4
2.6 Normalised Cross-Correlation 4
2.7 Statistical Moments and Hu Invariants 4
2.8 Logistic Regression 4
3. SYSTEM DESIGN AND ARCHITECTURE 5
3.1 Overall Architecture 5
3.2 Module Organisation 5
3.3 Dataset 6
3.4 Data Stored per Account 6
4. PREPROCESSING 7
4.1 Grayscale Conversion and Format Handling 7
4.2 Median Filtering and Contrast Stretch 7
4.3 Otsu Binarisation and Polarity Correction 7
4.4 Despeckling and Gap Bridging 8
4.5 Deskewing by Principal Axis 8
4.6 Cropping and Size Normalisation 8

5. FEATURE EXTRACTION 10
5.1 Projection Profiles as 1-D Signals 11
5.2 Energy, Entropy and Spectral Features 11
5.3 Two-Dimensional Spectrum 11
5.4 Haar Wavelet Decomposition 12
5.5 Contour Signals 12
6. MATCHING AND SCORING 13
6.1 Writer-Normalised Feature Distance 13
6.2 Peak Normalised Cross-Correlation 13
6.3 DSP Score Fusion 14
6.4 Writer-Independent Classifier 14
6.5 Scoring Modes 15
7. THRESHOLD CALIBRATION AND PARAMETER TUNING 16
7.1 Per-Writer Threshold by Leave-One-Out 16
7.2 Global Parameter Tuning 17
7.3 Classifier Training and Validation 18
8. GRAPHICAL USER INTERFACE 20
8.1 Layout 20
8.2 Activity Log 20
8.3 Loading States 20
8.4 Enrolment and Verification 21
8.5 Error and Warning States 21
8.6 DSP Analysis Figure 23
9. RESULTS AND EVALUATION 24
9.1 Evaluation Metrics 24
9.2 Results on Held-Out Writers 24
9.3 Score Distributions and ROC 25
9.4 Discussion 26
9.5 Example Decisions 26
10. APPLICATIONS 28
11. FUTURE SCOPE 28
12. CONCLUSION 29
REFERENCES 31

APPENDIX A: PROJECT FILE LISTING.....32
APPENDIX B: HOW TO RUN AND REPRODUCE.....32
APPENDIX C: SAMPLE ACTIVITY LOG.....33

LIST OF FIGURES

Figure : Block diagram of the signature verification system 5
Figure : Preprocessing stages on a CEDAR genuine signature.....7
Figure : Feature families and zoning densities of a genuine signature 10
Figure : DSP feature signals of a genuine signature (blue) and a skilled forgery (orange) 11
Figure : Leave-one-out threshold calibration for one CEDAR writer (hybrid mode) 17
Figure : Grid search over fusion weight and threshold tightness on training writers 18
Figure : Leave-one-writer-out probabilities of the classifier.....19
Figure : GUI on start-up, before any account is enrolled 21
Figure : Loading state: buttons disabled, amber status.....21
Figure : Genuine signature verified: AUTHENTIC 21
Figure : Skilled forgery: FORGED, 76.1 % confidence.....21
Figure : Warning: Verify pressed before enrolment.....22
Figure : Error: corrupt image, details in the log 22
Figure : Comparison of the three scoring modes on held-out writers 25
Figure : Distribution of hybrid scores for genuine and forged probes 25
Figure : FAR and FRR against threshold (left) and ROC curves of the three modes (right).....26
Figure : Analysis of a genuine test signature.....27
Figure : Analysis of a skilled forgery 27

LIST OF TABLES

Table : Main MATLAB modules and their roles 5
Table : Dataset split used in the experiments 6
Table : The nine feature families 10
Table : Scoring modes 15
Table : Tuned parameters (training writers 1 to 35).....18
Table : Verification results on 20 held-out CEDAR writers 24
Table : Project files and line counts.....32

1. INTRODUCTION

1.1 Background

Signature verification is a biometric problem: decide whether a questioned signature was produced by the person it claims to belong to. Two broad families of systems exist. *Online* systems capture the pen trajectory, pressure and timing on a digitising tablet. *Offline* systems work only with the static image of a signature on paper, which is the situation in a bank where cheques are scanned after they are written. Offline verification is harder because all dynamic information is lost, but it is the case that matters for cheque clearing.

Forgeries are usually grouped into three classes. A *random* forgery is a signature of a different person submitted by mistake or carelessly. A *simple* forgery is written by someone who knows the name but has not seen the real signature. A *skilled* forgery is produced by someone who has practised copying the genuine signature. Skilled forgeries are the hardest case and the one this project is evaluated against.

1.2 Problem Statement

Given three to six genuine specimen signatures collected when an account is opened, build a system that can examine any later scanned signature for that account and declare it AUTHENTIC or FORGED, together with a confidence value, while keeping both the false acceptance rate (FAR) and false rejection rate (FRR) low.

1.3 Objectives

- Implement a robust image preprocessing chain that removes noise and normalises position, scale and slant.
- Extract a compact, discriminative feature vector using DSP techniques: projection signals, FFT spectra, 2-D Fourier energy bands, Haar wavelet energies, moments and contour statistics.
- Design a writer-dependent matching score that respects each writer's natural variability, fused with 2-D normalised cross-correlation.
- Add a writer-independent machine learning classifier and compare DSP, ML and hybrid scoring modes.
- Calibrate the acceptance threshold per writer and tune global parameters on data kept separate from the test set.

- Provide a usable graphical interface with activity logging, loading indicators and clear error handling.
- Evaluate the system honestly on the public CEDAR database using writers never seen during training.

1.4 Scope of the Project

The system targets offline verification of a single signature image against a registered account. It runs on base MATLAB: every Image Processing Toolbox function it prefers has a local fallback, the Haar transform is implemented directly, and a built-in logistic regression replaces the SVM when the Statistics Toolbox is absent. It therefore also runs on GNU Octave. Signature localisation on a full cheque image and online (dynamic) verification are outside the scope.

2. THEORETICAL BACKGROUND

This chapter summarises the DSP concepts that the system relies on. Each is applied to the signature either as a 2-D image or as a 1-D signal derived from it.

2.1 Images as Two-Dimensional Signals

A grayscale image is a 2-D discrete signal $x[m, n]$ sampled on a rectangular grid. Every linear operation used here, such as smoothing or correlation, is a 2-D convolution of this signal with a small kernel $h[m, n]$:

$$y[m, n] = \Sigma_k \Sigma_l h[k, l] \cdot x[m - k, n - l]$$

Non-linear operators such as the median filter and thresholding are also applied pixel by pixel over a neighbourhood. Summing the image along rows or columns turns it into a 1-D signal that can be analysed with standard 1-D tools.

2.2 Median Filtering

Scanned documents contain impulse (salt-and-pepper) noise from dust and sensor defects. A 3×3 median filter replaces each pixel with the median of its nine neighbours. Unlike a mean filter it removes isolated impulses without blurring stroke edges, which is why it is preferred for line drawings such as signatures.

2.3 Otsu Thresholding

To separate ink from paper the image must be binarised. Otsu's method chooses the global threshold t that maximises the between-class variance of the two resulting classes:

$$\sigma_B^2(t) = [\mu_T \cdot \omega(t) - \mu(t)]^2 / (\omega(t) \cdot [1 - \omega(t)])$$

where $\omega(t)$ is the cumulative probability of the histogram up to level t, $\mu(t)$ is the cumulative mean and μ_T the global mean. The threshold is computed from the data, so it adapts to light pencil and dark ballpoint ink alike.

2.4 Discrete Fourier Transform

The DFT of an N-point signal is $X[k] = \Sigma\, x[n] \cdot e^{(-j2\pi kn/N)}$, computed efficiently by the FFT. Its magnitude describes how much energy the signal has at each frequency and is invariant to circular shifts of the signal. Applied

to the column projection of a signature, the low harmonics capture the quasi-periodic rhythm with which a person forms letters. The 2-D FFT of the whole image describes the orientation and fineness of strokes.

2.5 Discrete Wavelet Transform (Haar)

The Haar wavelet splits an image into a low-pass approximation and three detail sub-bands (horizontal, vertical, diagonal) by pairwise sums and differences scaled by $1/\sqrt{2}$. Repeating the split on the approximation gives a multiresolution decomposition. The energy in each sub-band summarises how much stroke detail exists at each scale and direction.

2.6 Normalised Cross-Correlation

Cross-correlation measures how similar two signals are as one is shifted over the other. Normalising by the energies of both images yields a value in [-1, 1] that is independent of ink density. By the correlation theorem it is computed in the frequency domain:

$$R = IFFT2(\, conj(FFT2(A)) \cdot FFT2(B)\,), \qquad NCC = max(R) / (\, ||A|| \cdot ||B||\,)$$

Taking the peak over all shifts makes the measure tolerant to small displacements between the reference and the questioned signature.

2.7 Statistical Moments and Hu Invariants

Central moments μ_{pq} describe the spatial distribution of ink. Hu derived seven combinations of normalised central moments that are invariant to translation, scale and rotation. They summarise the global shape of the signature in seven numbers.

2.8 Logistic Regression

Logistic regression models the probability of a binary label as $p = 1 / (1 + e^{-(w \cdot x + b)})$. Weights are found by minimising the cross-entropy loss with gradient descent. With L2 regularisation and class weighting it is a simple, well-calibrated classifier that needs no toolbox.

3. SYSTEM DESIGN AND ARCHITECTURE

3.1 Overall Architecture

The system is split into two phases, shown in Figure 1. During **enrolment**, which happens once when an account is opened, the specimen signatures are preprocessed, converted to feature vectors and summarised into a statistical model of that writer. During **verification**, each questioned signature passes through the same preprocessing and feature extraction, is compared with the stored model, and receives a fused score that is tested against the writer's own calibrated threshold.

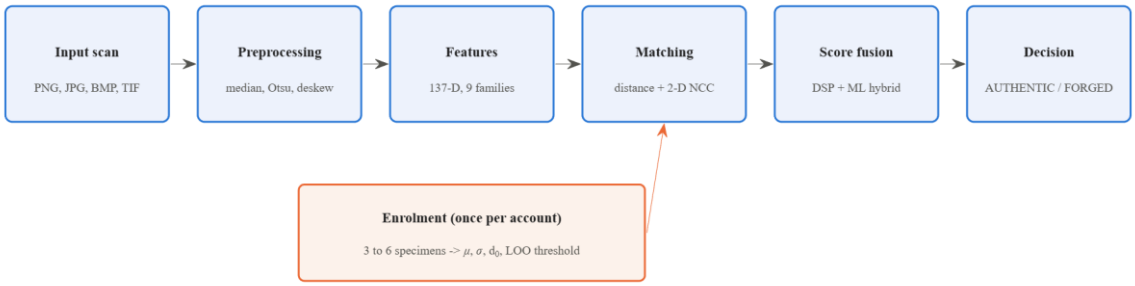


Figure 1: Block diagram of the signature verification system

3.2 Module Organisation

The MATLAB code is organised so that each DSP stage lives in its own function. The top-level scripts and the GUI only orchestrate these functions. Table 1 lists the main modules.

Table 1: Main MATLAB modules and their roles

File	Role
src/preprocessSignature.m	Grayscale, median filter, Otsu, despeckle, deskew, crop, resize
src/extractFeatures.m	137-D feature vector across nine DSP families
src/enrollUser.m	Builds the writer model and calibrates the threshold
src/signatureScore.m	Feature distance, cross-correlation and score fusion
src/verifySignature.m	AUTHENTIC / FORGED decision and analysis plots
src/looScores.m, classStats.m	Leak-free leave-one-out statistics
trainClassifier.m	Writer-independent classifier on pair vectors

File	Role
tuneParameters.m	Grid search of fusion weights and threshold tightness
main_signature_verification.m	Full evaluation: FAR, FRR, EER, ROC
registerPerson.m, checkSignature.m	Bank workflow: register an account, verify a cheque
signatureGUI.m	Interactive front end with activity log

3.3 Dataset

All reported results use the **CEDAR** signature database from the University at Buffalo. It contains 55 writers, each with 24 genuine signatures and 24 skilled forgeries, scanned at 300 dpi in grayscale. The helper `importCEDAR` converts it into the project layout, one folder per writer with files named `genuine_NN.png` and `forged_skilled_NN.png`.

The split is made **by writer**, not by image. Writers 1 to 35 are used for training the classifier and tuning the parameters; writers 36 to 55 are held out and used only for the final evaluation. No test writer is ever seen during training or tuning, which is the situation a bank faces when a new customer opens an account.

Table 2: Dataset split used in the experiments

Subset	Writers	Use	Images
Training / tuning	1 to 35	Classifier, grid search	35 × 48 = 1680
Held-out test	36 to 55	Final evaluation only	20 × 48 = 960
Per test writer	1	6 references, 18 genuine probes, 24 forgeries	48

3.4 Data Stored per Account

When an account is registered, `registerPerson` saves a model file `models/<accountId>.mat`. It holds the 137 × N feature matrix of the specimens, the per-feature mean and standard deviation (the writer's natural variability), the intra-class distance scale *d0*, the preprocessed specimen images needed by the cross-correlation term, the scoring mode and the calibrated threshold. Nothing else about the customer is required at verification time.

4. PREPROCESSING

Preprocessing turns an arbitrary scan into a clean, binary, normalised image so that later stages compare the signature itself and not the scanner, the pen or the position on the page. The stages, implemented in `preprocessSignature.m`, are illustrated on a real CEDAR sample in Figure 2.

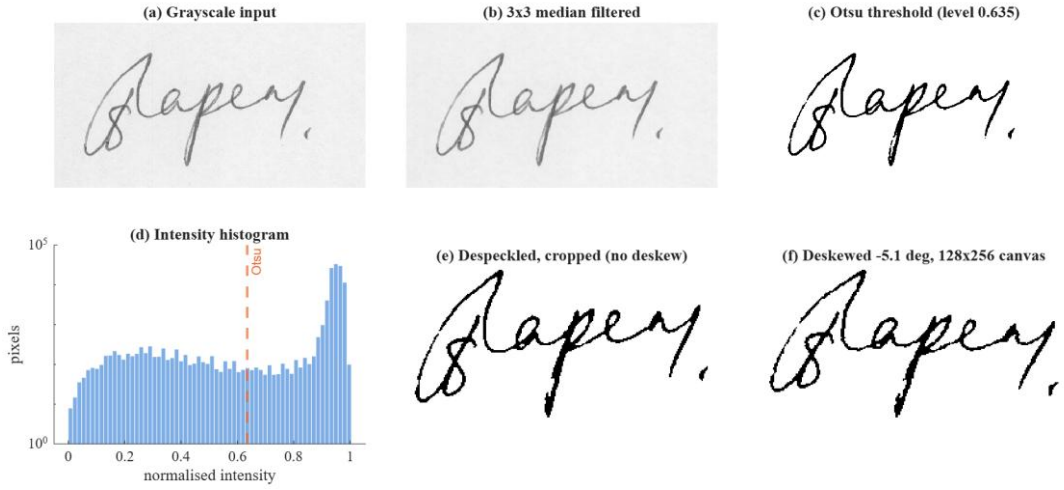


Figure 2: Preprocessing stages on a CEDAR genuine signature

4.1 Grayscale Conversion and Format Handling

The input may be RGB, indexed (palette) PNG, logical or grayscale. Indexed images are expanded through their colour map, RGB images are converted with the luminance weights 0.2989 R + 0.5870 G + 0.1140 B, and the result is scaled to the range [0, 1].

4.2 Median Filtering and Contrast Stretch

A 3×3 median filter removes scanner dust and isolated noise pixels (Figure 2b). A linear contrast stretch then maps the darkest pixel to 0 and the lightest to 1, so that faint pencil and dark gel ink occupy the same dynamic range before thresholding.

4.3 Otsu Binarisation and Polarity Correction

The stretched image is thresholded at the Otsu level computed from its 256-bin histogram (Figure 2c and 2d). The histogram is clearly bimodal: a large paper peak near 1 and a broad ink distribution below. If more than half of the result is foreground the polarity is inverted, so ink is always 1 regardless of whether the scan was dark-on-light or light-on-dark.

4.4 Despeckling and Gap Bridging

Connected components smaller than 20 pixels are removed, which deletes stray dots that survived the median filter. A morphological closing with a disk of radius 1 then bridges one-pixel breaks in strokes caused by uneven ink flow.

4.5 Deskewing by Principal Axis

Signatures are rarely written perfectly horizontally. The slant angle is estimated from the second-order central moments of the ink pixels:

$$\theta = \tfrac{1}{2} \cdot \operatorname{atan2}(2\mu_{11}, \mu_{20} - \mu_{02})$$

The image is rotated by θ (limited to ±30 degrees, and only when $|\theta| > 1$ degree) so that the principal axis of the signature is horizontal (Figure 2f). This makes the projection profiles of different samples comparable.

4.6 Cropping and Size Normalisation

The image is cropped to the bounding box of the ink and resized, preserving its aspect ratio, to fit inside a 128 × 256 canvas with a 4-pixel margin, centred. Translation and scale invariance therefore come directly from this stage. The relevant part of the implementation is shown below.

Listing 4.1: Core of preprocessSignature.m

```
G = medianFilter(G, opts.medianSize);


lo = min(G(:)); hi = max(G(:));
if hi - lo > eps
    G = (G - lo) / (hi - lo);
end


level = otsuLevel(G);
B = G > level;


if mean(B(:)) > 0.5
    B = ~B;
end
info.otsuLevel = level;


B = despeckle(B, opts.minBlob);
B = bridgeGaps(B);


if ~any(B(:))
    warning('preprocessSignature:empty', 'No ink found after thresholding.');
    B = false(opts.size);
    info.angle = 0; info.bbox = [1 1 1 1]; info.inkPixels = 0;
    info.aspectRaw = 1;
    return;
end


angle = 0;
```

```
if opts.deskew
    [r, c] = find(B);
    r = r - mean(r); c = c - mean(c);
    mu20 = mean(c.^2); mu02 = mean(r.^2); mu11 = mean(r.*c);
    angle = 0.5 * atan2(2*mu11, mu20 - mu02) * 180/pi;
    angle = max(min(angle, 30), -30);
    if abs(angle) > 1
        B = rotateImg(B, angle);
        B = despeckle(B, opts.minBlob);
    end
end
info.angle = angle;
```

5. FEATURE EXTRACTION

Each preprocessed signature is reduced to a 137-element feature vector by `extractFeatures.m`. The features are grouped into nine families, each capturing a different aspect of the handwriting. Figure 3 shows how the 137 dimensions are distributed and one of the simplest families, the 4×4 zoning grid.

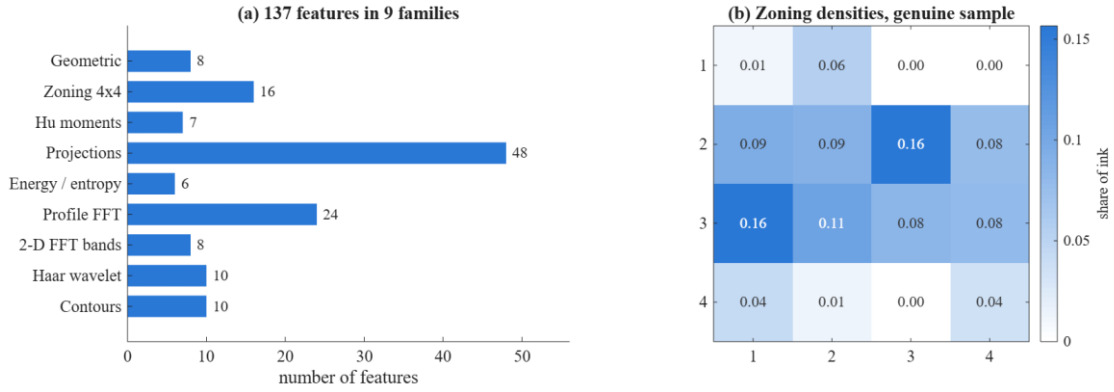


Figure 3: Feature families and zoning densities of a genuine signature

Table 3: The nine feature families

Family	Count	What it captures
Geometric / statistical	8	Ink density, centroid, spread, aspect ratio, stroke thickness, loop count
Zoning (4 × 4)	16	Local distribution of ink across the canvas
Hu invariant moments	7	Global shape, invariant to translation, scale, rotation
Projection profiles	48	Column (32) and row (16) ink sums resampled as 1-D signals
Signal energy / entropy	6	Energy, Shannon entropy, first-difference energy of both profiles
Profile FFT magnitude	24	16 + 8 low harmonics: rhythm of letter formation
2-D FFT radial bands	8	Stroke texture and fineness from the image spectrum
Haar wavelet energies	10	Multiresolution detail, 3 levels × 3 orientations + approximation
Contour signals	10	Mean, std, skewness, kurtosis, roughness of upper and lower envelopes

5.1 Projection Profiles as 1-D Signals

Summing the binary image down each column gives the vertical projection *p_v[n]*, and summing along each row gives the horizontal projection *p_h[m]*. These two signals carry most of the structure of a signature: where the tall letters are, how dense each part is and how the baseline drifts. Figure 4a and 4b compare them for a genuine signature and a skilled forgery of the same name. The forger reproduces the overall envelope but not the exact positions and heights of the peaks.

5.2 Energy, Entropy and Spectral Features

For each normalised profile the system computes its mean energy, its Shannon entropy (how evenly ink is spread) and the energy of its first difference (how abruptly ink density changes). The FFT of the mean-removed profile is then taken and the magnitudes of the first 16 (vertical) and 8 (horizontal) harmonics are kept, normalised to unit sum. Because the FFT magnitude is shift invariant, these features are robust to small horizontal misalignment, while still describing the writer's rhythm (Figure 4c).

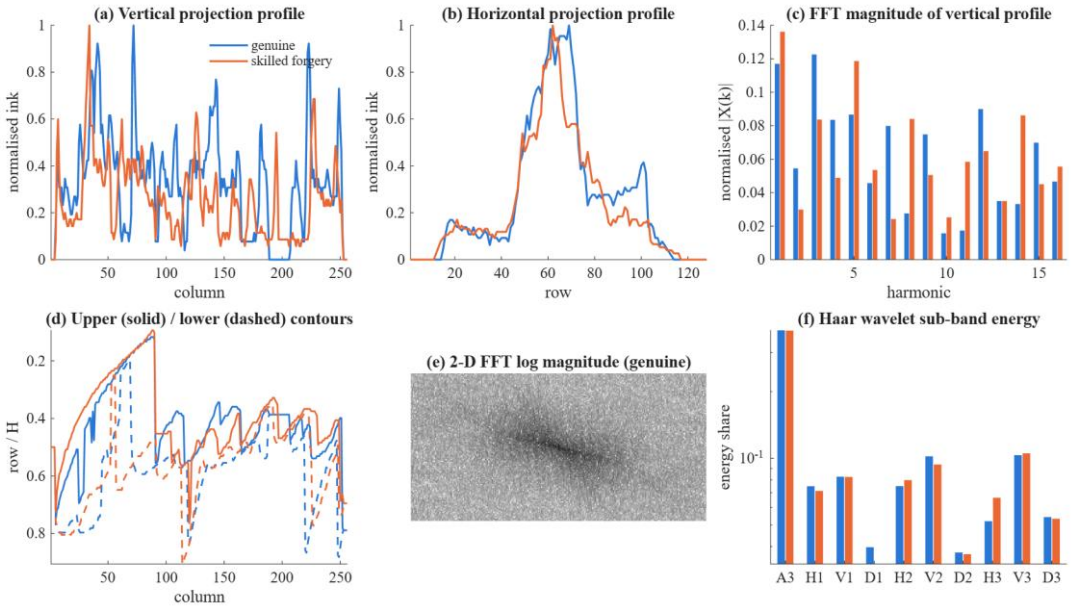


Figure 4: DSP feature signals of a genuine signature (blue) and a skilled forgery (orange)

5.3 Two-Dimensional Spectrum

The magnitude of the 2-D FFT of the whole image (Figure 4e) is split into eight concentric radial bands and the energy in each band is recorded. Low bands describe the large-scale shape, high bands the stroke width and fine

detail. The dominant diagonal orientation in the spectrum reflects the slant of the strokes.

5.4 Haar Wavelet Decomposition

A three-level 2-D Haar transform, implemented directly without the Wavelet Toolbox, produces one approximation and nine detail sub-bands. Their normalised energies (Figure 4f) form ten features. The level-1 diagonal band in particular differs between genuine and forged samples because forgers write more slowly and produce smoother, less jagged strokes.

Listing 5.1: One level of the 2-D Haar transform

```
function [A, Hd, Vd, Dd] = haarStep(X)
[h, w] = size(X);
if mod(h,2), X(end+1,:) = 0; end
if mod(w,2), X(:,end+1) = 0; end
s  = 1/sqrt(2);
L  = s*(X(1:2:end,:) + X(2:2:end,:));
Hh = s*(X(1:2:end,:) - X(2:2:end,:));
A  = s*(L(:,1:2:end)  + L(:,2:2:end));
Vd = s*(L(:,1:2:end)  - L(:,2:2:end));
Hd = s*(Hh(:,1:2:end) + Hh(:,2:2:end));
Dd = s*(Hh(:,1:2:end) - Hh(:,2:2:end));
end
```

5.5 Contour Signals

For every column the first and last ink pixel define the upper and lower contour of the signature (Figure 4d). Their mean, standard deviation, skewness, kurtosis and roughness (mean absolute first difference) give ten features describing the outline of the signature.

6. MATCHING AND SCORING

A questioned signature is compared with the stored model in two independent ways, one in the feature domain and one in the image domain, and the two similarities are fused. An optional machine learning probability is then combined with this DSP score.

6.1 Writer-Normalised Feature Distance

People differ in how consistently they sign. A feature that varies a lot between one writer's genuine samples should count for less when judging that writer. Each feature difference is therefore divided by that writer's own standard deviation σ_i (with a small floor to avoid division by zero), and the root-mean-square z-score distance to each reference r is computed:

$$d_r = sqrt((1/137) \cdot \Sigma_i ((f_i - F_ir) / \sigma_i)^2)$$

The median of these distances over all references is mapped to a similarity in (0, 1] using the writer's mean intra-class distance $d0$:

$$featureSim = exp(- median(d_r) / d0)$$

A test signature that is as close to the references as they are to each other scores about 0.37; a much closer one approaches 1.

6.2 Peak Normalised Cross-Correlation

The second measure compares the images directly. Both binary images are first smoothed with a 7×7 Gaussian kernel (σ = 1.6) so that strokes displaced by a pixel or two still overlap. The normalised cross-correlation is computed in the frequency domain with the FFT, and its peak over all shifts is taken. The median over all references gives *corrSim*.

Listing 6.1: FFT-based peak normalised cross-correlation

```
function c = peakNCC(A, B)
A = smoothInk(double(A));
B = smoothInk(double(B));


A = A - mean(A(:));
B = B - mean(B(:));
na = sqrt(sum(A(:).^2)); nb = sqrt(sum(B(:).^2));
if na <= eps || nb <= eps, c = 0; return; end


R = real(ifft2(conj(fft2(A)) .* fft2(B)));
c = max(R(:)) / (na * nb);
c = min(max(c, 0), 1);
end
```

6.3 DSP Score Fusion

The two similarities are combined linearly:

$$dspScore = w1 \cdot featureSim + w2 \cdot corrSim$$

The default weights were 0.6 and 0.4. After tuning on the training writers (Chapter 7) the weights became 0.2 and 0.8, showing that on real scans the correlation term carries more reliable information than the feature distance.

Listing 6.2: Feature distance and DSP fusion in signatureScore.m

```
D = bsxfun(@minus, F, f);
D = bsxfun(@rdivide, D, sigma);
d = sqrt(mean(D.^2, 1));


dRef = median(d);


featureSim = exp(-dRef / max(model.d0, eps));


corrSim = 0;
if ~isempty(B) && isfield(model, 'refImgs')
    idx = find(keep);
    cvals = zeros(1, numel(idx));
    for k = 1:numel(idx)
        cvals(k) = peakNCC(model.refImgs{idx(k)}, B);
    end
    corrSim = max(0, median(cvals));
end


w = model.weights;
dsp = min(max(w(1)*featureSim + w(2)*corrSim, 0), 1);
```

6.4 Writer-Independent Classifier

The DSP score is *writer-dependent*: nothing is learned across people. To add learning without requiring retraining for every new customer, the problem is reformulated as a pair classification. For every (enrolled model, questioned signature) pair a 142-D **difference vector** is built: the 137 per-feature z-scores $|f - \mu| / \sigma$, plus the minimum, median and maximum reference distance, featureSim and corrSim. Label 1 means a genuine pair, 0 a forged pair.

A single classifier learns how genuine pairs differ from forged pairs in general. An RBF support vector machine is used when the Statistics Toolbox is available; otherwise the project trains its own L2-regularised logistic regression with class-balanced weights ($\lambda = 0.01$, learning rate 0.1, 5000 gradient descent iterations). The second option was used for all reported results.

6.5 Scoring Modes

Table 4: Scoring modes

Mode	Final score
dsp	w1 · featureSim + w2 · corrSim
ml	classifier probability p
hybrid	0.5 · dspScore + 0.5 · p (default when a classifier exists)

7. THRESHOLD CALIBRATION AND PARAMETER TUNING

7.1 Per-Writer Threshold by Leave-One-Out

A single global threshold would be too strict for erratic signers and too lenient for very consistent ones. The system instead calibrates a threshold for every account at enrolment time, using only that account's specimens. Each specimen in turn is held out and scored against the remaining ones, giving N leave-one-out (LOO) genuine scores. The threshold is placed *k* standard deviations below their mean:

$$thr = mean(LOO) - k \cdot std(LOO), \quad \text{clamped to } [0.05, 0.95]$$

An important detail is that the writer statistics σ and d0 are **rebuilt from the remaining specimens** each time. Using statistics computed with the held-out sample included would make it look closer than a truly new signature, and the threshold would be set too high. This leak-free LOO is implemented in `looScores.m`.

Listing 7.1: looScores.m

```
function out = looScores(model)


N = size(model.F, 2);
for i = 1:N
    keep = true(1, N); keep(i) = false;
    sub = model;
    sub.F = model.F(:, keep);
    sub.refImgs = model.refImgs(keep);
    [sub.sigma, sub.d0] = classStats(sub.F);
    s = signatureScore(sub, model.F(:, i), model.refImgs{i});
    if i == 1
        out = s;
    else
        out(i) = s;
    end
end
end
```

Listing 7.2: Threshold calculation in enrollUser.m

```
L = looScores(model);
loo = [L.score];
thr = mean(loo) - opts.kThresh * std(loo);
thr = min(max(thr, opts.minThr), opts.maxThr);
```

Figure 5 shows the calibration for one CEDAR writer in hybrid mode. The six green points are the LOO scores of the enrolment specimens, giving a threshold of 0.607. Of the 24 skilled forgeries, 23 fall below it. The genuine

probes show how much this writer's signature varies from day to day: 10 of 18 are accepted, which illustrates why FRR is the harder error to control for inconsistent writers.

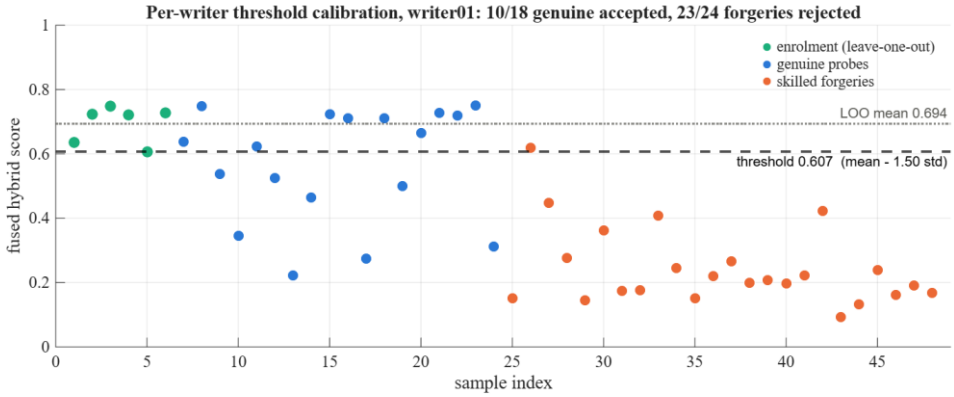


Figure 5: Leave-one-out threshold calibration for one CEDAR writer (hybrid mode)

7.2 Global Parameter Tuning

Only two kinds of numbers in the system are set by hand: the fusion weights (w1, w2) and the threshold tightness *k*. These are tuned by `tuneParameters.m` with an exhaustive grid search on the **training writers only**, so the final evaluation stays honest.

- Feature weight w1 from 0 to 1 in steps of 0.1 (w2 = 1 - w1): 11 values.
- Threshold tightness k from 0 to 6 in steps of 0.25: 25 values.
- For every grid point the per-writer thresholds are recomputed and FAR, FRR and the half total error rate HTER = (FAR + FRR) / 2 are measured.
- The threshold-free equal error rate (EER) is also recorded for each weight setting.

Score components are computed once per writer and cached, so the sweep over 275 grid points takes seconds. The ML and hybrid modes are then tuned separately for their own *k*. Figure 6 shows the HTER surface and the EER as a function of the fusion weight. The selected values are given in Table 5.

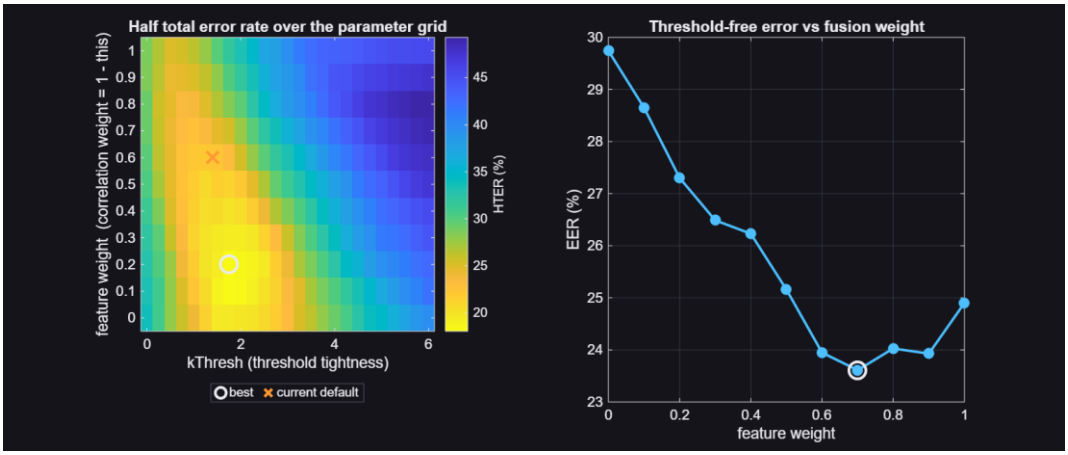


Figure 6: Grid search over fusion weight and threshold tightness on training writers

Table 5: Tuned parameters (training writers 1 to 35)

Parameter	Value
Fusion weights (feature / correlation)	0.2 / 0.8
k, DSP mode	1.75
k, ML mode	1.50
k, hybrid mode	1.50
Threshold clamp	[0.05, 0.95]
Reference specimens per writer	6

7.3 Classifier Training and Validation

The classifier is trained on 1680 pairs from the 35 training writers (840 genuine, 840 forged). Its quality is measured by **leave-one-writer-out** cross-validation: the classifier is trained on 34 writers and tested on the remaining one, for all 35 folds. This is the honest estimate for a writer-independent model. The LOWO equal error rate is 19.76 % and the accuracy at a 0.5 cut-off is 79.58 %. Figure 7 shows the distribution of out-of-fold probabilities.

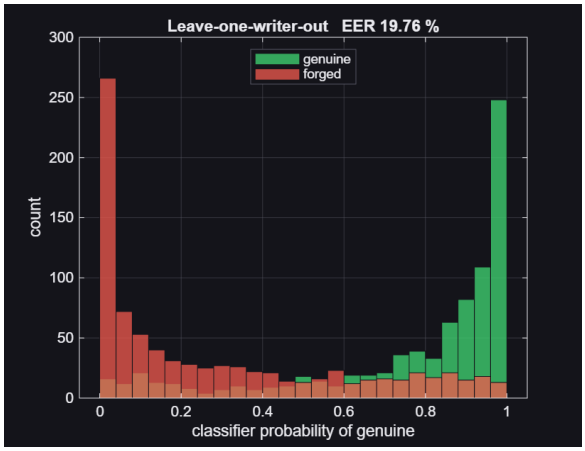


Figure 7: Leave-one-writer-out probabilities of the classifier

8. GRAPHICAL USER INTERFACE

For demonstration and for use by bank staff, `signatureGUI.m` provides a single-window interface built with MATLAB App Designer components (`uifigure`, `uigridlayout`, `uigauge`). The workflow follows three numbered buttons: enrol a writer from a folder of specimens, load the questioned signature, and verify. A fourth button opens the detailed DSP analysis figure.

8.1 Layout

The window is divided into five rows: a header with an account status chip, two image panels for the reference specimen and the questioned signature, a decision panel, an activity log, and the action bar (Figure 8). The decision panel contains a circular gauge whose red and green zones are split exactly at the account's threshold, the verdict in large type, a horizontal score bar with the threshold marked, and a table of seven metrics: fused score, DSP score, ML probability, threshold, feature similarity, cross-correlation and confidence.

8.2 Activity Log

Every action is written to an on-screen activity log and appended to `logs/signatureGUI.log` with a date and time stamp. The newest entry is shown at the top. Each entry carries a level: INFO for normal steps, OK for successful operations, ALERT when a signature is judged forged, WARN for user mistakes and ERROR for failures. The log records which parameters and classifier were used, enrolment time, the LOO scores, the threshold, preprocessing details of each loaded image (Otsu level, deskew angle, ink pixel count) and the full result of every verification. This gives an audit trail of the kind a bank would require.

8.3 Loading States

Enrolment, image loading, verification and the analysis figure can each take from a fraction of a second to several seconds. While any of them runs, a helper function `busy` disables all four buttons so the operation cannot be started twice, shows an indeterminate progress dialog with a description of the step, and turns the status line amber (Figure 9). The returned `onCleanup` object guarantees that the buttons are re-enabled and the dialog closed even if the operation fails.

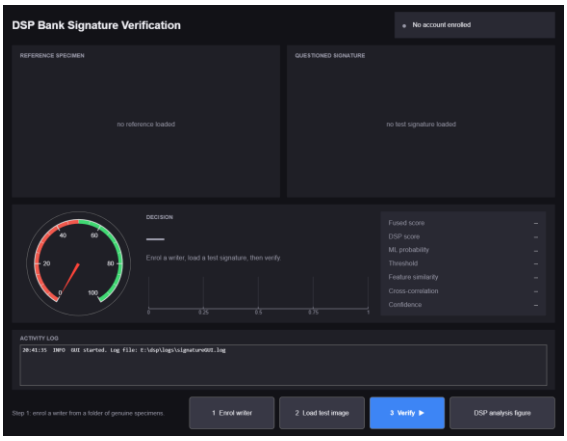


Figure 8: GUI on start-up, before any account is enrolled

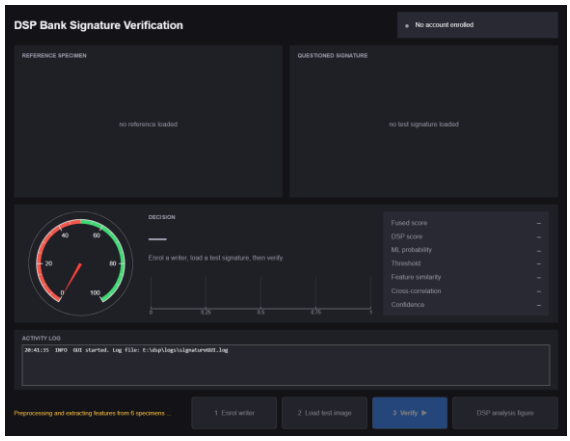


Figure 9: Loading state: buttons disabled, amber status

8.4 Enrolment and Verification

After enrolment the reference panel shows the first preprocessed specimen, the caption gives the number of specimens and the calibrated threshold, and the header chip turns green with the account name and scoring mode. Loading and verifying a genuine signature produces the AUTHENTIC result in Figure 10; the questioned signature is drawn in blue. A skilled forgery produces the FORGED result in Figure 11, drawn in orange, with the low ML probability (0.033) and feature similarity clearly visible in the metric table.

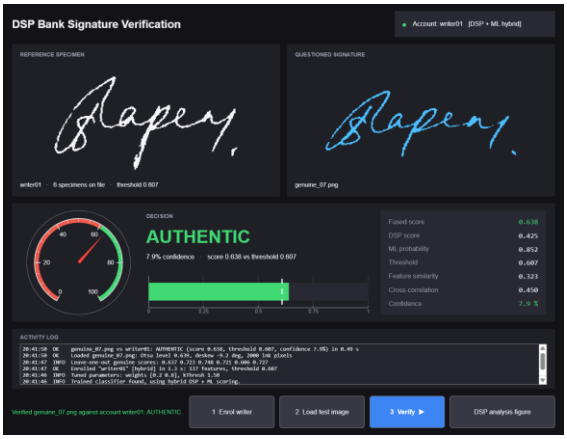


Figure 10: Genuine signature verified: AUTHENTIC

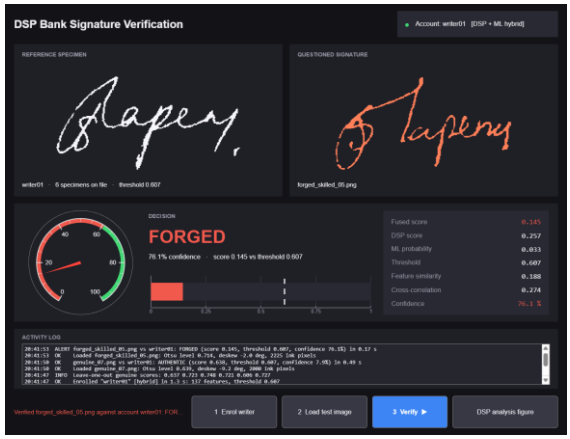


Figure 11: Skilled forgery: FORGED, 76.1 % confidence

8.5 Error and Warning States

Two kinds of problems are distinguished. **Warnings** are user mistakes, such as pressing Verify before enrolling a writer, choosing a folder with fewer than

three images, or loading a blank scan. They show a warning dialog, an amber status line and a WARN log entry (Figure 12). **Errors** are failures inside the processing, such as an unreadable or corrupt image file. The error message, its MATLAB identifier and the function and line where it occurred are written to the log, the status line turns red, an error dialog is shown and the affected panel is reset so no stale verdict remains on screen (Figure 13). If enrolment fails, the account chip turns red and the previously enrolled model, if any, is kept.

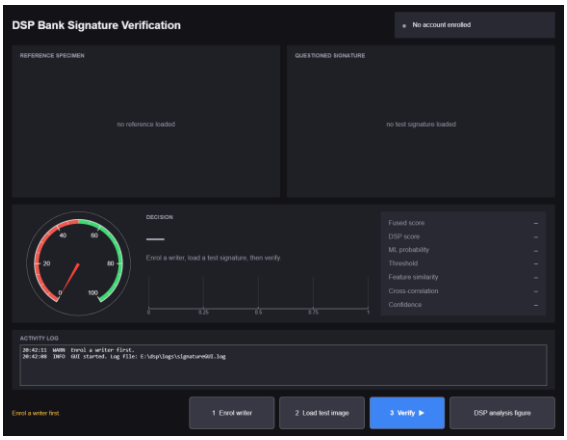


Figure 12: Warning: Verify pressed before enrolment

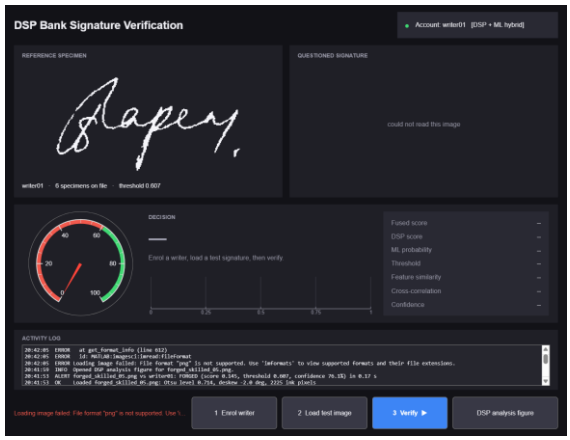


Figure 13: Error: corrupt image, details in the log

Listing 8.1: Logging, loading and error helpers in signatureGUI.m

```
    % Loading state: lock the buttons, show a progress dialog and an amber
    % status line. Everything is restored when the returned object dies.
    function c = busy(title, msg)
        btns = [btnEnrol btnLoad btnVerify btnDetail];
        set(btns, 'Enable', 'off');
        setStatus([msg ' ...'], CBUSY);
        dlg = uiprogressdlg(fig, 'Title', title, 'Message', msg, ...
                            'Indeterminate', 'on');
        drawnow;
        c = onCleanup(@() unbusy(btns, dlg));
    end


    function unbusy(btns, dlg)
        if isvalid(dlg), close(dlg); end
        if isvalid(fig), set(btns, 'Enable', 'on'); end
    end


    % Error state: red status line, full error in the log, alert dialog.
    function reportError(where, ME)
        logMsg('ERROR', '%s failed: %s', where, ME.message);
        if ~isempty(ME.identifier), logMsg('ERROR', '  id: %s', ME.identifier); end
        if ~isempty(ME.stack)
            logMsg('ERROR', '  at %s (line %d)', ME.stack(1).name, ME.stack(1).line);
        end
        setStatus(sprintf('%s failed: %s', where, ME.message), CBAD);
        uialert(fig, ME.message, [where ' failed'], 'Icon', 'error');
    end
```

8.6 DSP Analysis Figure

The analysis button opens a separate figure, of the same kind as Figures 17 and 18 in Chapter 9, that explains the decision visually: the reference and questioned images, a colour overlay in which yellow marks overlapping ink, the vertical projection profiles of both, the FFT harmonics of those profiles and the score bar against the threshold.

9. RESULTS AND EVALUATION

9.1 Evaluation Metrics

- **False Rejection Rate (FRR):** percentage of genuine signatures wrongly rejected.
- **False Acceptance Rate (FAR):** percentage of forgeries wrongly accepted. For a bank this is the costlier error.
- **Accuracy:** percentage of all decisions that are correct at the operating point.
- **Equal Error Rate (EER):** the error rate at the global threshold where FAR equals FRR. It measures how well the scores separate the two classes regardless of any threshold.

FAR, FRR and accuracy are measured with the per-writer calibrated thresholds, exactly as the system would operate in a bank. EER is computed from a sweep of a single global threshold from 0 to 1 in 501 steps.

9.2 Results on Held-Out Writers

All three modes were evaluated on the 20 held-out CEDAR writers: 360 genuine probes and 480 skilled forgeries. Table 6 and Figure 14 summarise the results.

Table 6: Verification results on 20 held-out CEDAR writers

Mode	FRR %	FAR %	Accuracy %	EER %
DSP	18.33	11.46	85.60	23.54
ML	18.89	16.46	82.50	16.67
Hybrid	18.89	12.71	84.64	16.56

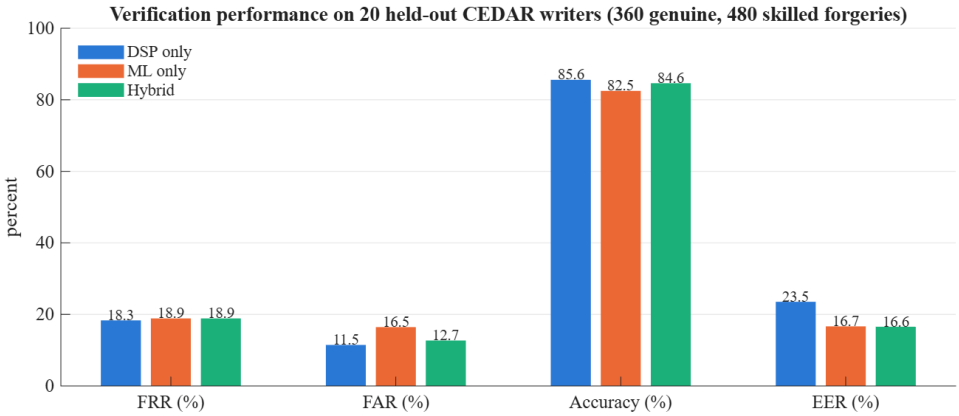


Figure 14: Comparison of the three scoring modes on held-out writers

9.3 Score Distributions and ROC

Figure 15 shows the histogram of hybrid scores for genuine and forged probes. The two populations are clearly shifted apart, with overlap in the middle range caused by inconsistent genuine samples and the best forgeries. Figure 16 plots FAR and FRR against the global threshold and the ROC curves of all three modes. The ML and hybrid curves lie above the DSP curve over most of the range.

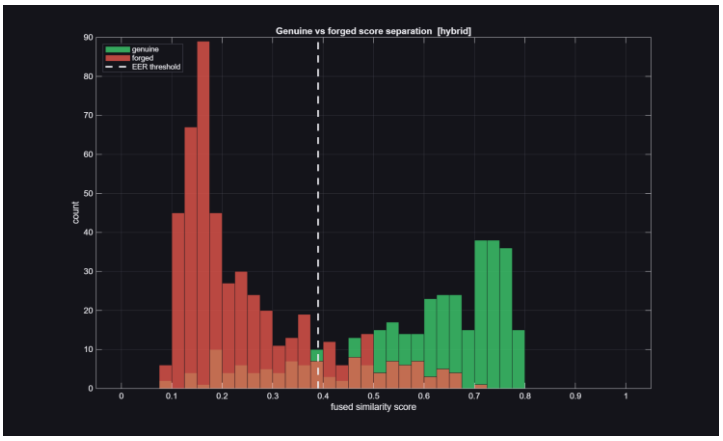


Figure 15: Distribution of hybrid scores for genuine and forged probes

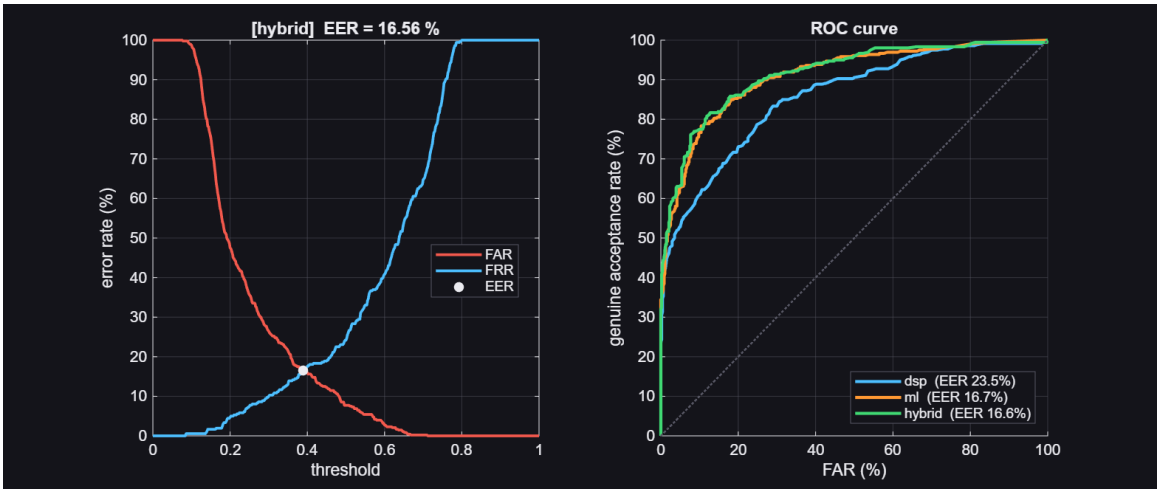


Figure 16: FAR and FRR against threshold (left) and ROC curves of the three modes (right)

9.4 Discussion

- **Separability:** the ML and hybrid modes reduce the EER from 23.5 % to about 16.6 %, a relative improvement of roughly 30 %. The learned classifier is clearly better at ranking genuine above forged signatures.
- **Operating point:** with per-writer thresholds all three modes reach 82 to 86 % accuracy. DSP-only keeps a slight edge in accuracy because its cross-correlation term lives on a writer-specific scale that the per-writer threshold exploits directly.
- **Hybrid:** combines near-best accuracy with the best EER, giving the bank much more room to move the FAR/FRR trade-off by adjusting k. It is therefore the default mode in the GUI and in `registerPerson`.
- **Speed:** enrolment of six specimens takes about 1.3 s and each verification 0.2 to 0.5 s on a laptop without any toolboxes, which is fast enough for interactive use at a bank counter.
- **Honesty of the numbers:** no test writer was used for training or tuning, and LOO statistics are leak-free. Published systems with deep networks reach lower error rates on CEDAR, but they are trained on far more data; the result here is reasonable for a compact, explainable, toolbox-free DSP system.

9.5 Example Decisions

Figures 17 and 18 are the analysis figures for one genuine and one skilled forgery from the evaluation run. In the genuine case the overlay is largely

yellow and the projection profiles align; in the forged case the strokes overlap only in places and the FFT harmonics differ.

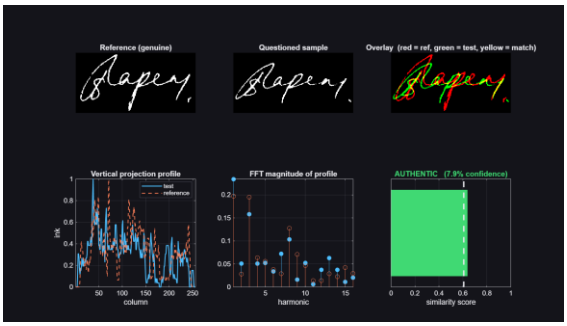


Figure 17: Analysis of a genuine test signature

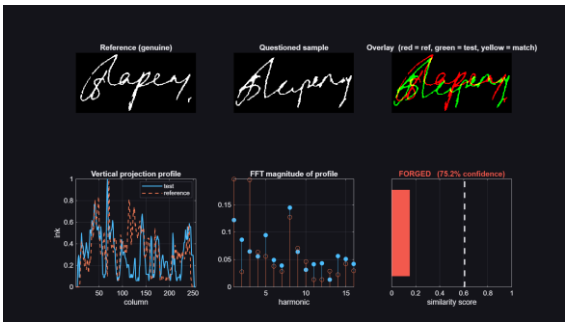


Figure 18: Analysis of a skilled forgery

10. APPLICATIONS

- **Cheque clearing:** automatic first-pass verification of signatures on scanned cheques in cheque truncation systems, with only low-confidence cases sent to a human verifier.
- **Branch counters:** verification of withdrawal slips, demand draft requests and account modification forms against the specimen held on file.
- **Account opening and KYC:** checking that signatures on different documents submitted by the same customer are consistent.
- **Legal and government documents:** screening of signatures on affidavits, contracts, land records and certificates.
- **Examination and academic records:** verifying candidate signatures on answer sheets and attendance registers.
- **Insurance and loan processing:** detecting forged signatures on claim forms and loan agreements.
- **Forensic document examination:** providing an objective similarity score and visual analysis to support expert opinion.
- **Teaching:** a compact, readable example of how filtering, FFT, wavelets and correlation combine in a real application.

11. FUTURE SCOPE

- **Deep learning features:** a Siamese convolutional network trained on larger databases such as GPDS or MCYT could replace or complement the hand-crafted features and reduce the EER further.
- **More data:** evaluating on additional public databases and on real cheque scans from a partner bank to measure performance across scanners, inks and languages.
- **Signature localisation:** automatically finding and cropping the signature region on a full cheque image before verification.
- **Local features:** adding local binary patterns, histogram of oriented gradients or SIFT-like keypoints to capture fine stroke detail.

- **Dynamic time warping:** aligning projection and contour signals elastically before comparing them, to tolerate natural stretching of the signature.
- **Template ageing:** updating a customer's model with newly verified signatures so that gradual changes in handwriting over years are tracked.
- **Risk-based thresholds:** using a stricter k for high-value transactions and a looser one for low-value ones.
- **Deployment:** packaging the system as a MATLAB Production Server service or compiling it with MATLAB Coder so it can be integrated with core banking software, with the activity log forwarded to a central audit system.
- **Online signatures:** extending the system to tablets and signature pads where pen trajectory and pressure are also available.

12. CONCLUSION

This project designed, implemented and evaluated a complete offline signature verification system for banking based on digital signal processing. A scanned signature is cleaned with a median filter, binarised with Otsu's method, deskewed using image moments and normalised in size. It is then described by 137 features drawn from nine DSP families, including projection signals and their FFT spectra, 2-D Fourier energy bands, Haar wavelet energies, Hu moments and contour statistics.

Matching combines a writer-normalised feature distance with FFT-based normalised cross-correlation, and a writer-independent logistic regression classifier adds learning on top of the same features. Thresholds are calibrated per writer with a leak-free leave-one-out procedure, and the remaining global parameters are tuned by grid search on training writers only.

On 20 held-out writers of the CEDAR database the system reaches 85.6 % accuracy in DSP mode and an equal error rate of 16.6 % in hybrid mode, against skilled forgeries, without any MATLAB toolbox. A graphical interface with an activity log, loading indicators and clear warning and error states makes the system usable by non-specialist staff and leaves an audit trail.

The project shows that classical signal processing, applied carefully and evaluated honestly, gives an explainable and efficient verification system, and that a small amount of machine learning on the same DSP features improves its discriminating power considerably. The future scope outlined above, especially deep learning features and real cheque data, offers a clear path to production-grade accuracy.

REFERENCES

[1] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62-66, 1979.

[2] M.-K. Hu, "Visual pattern recognition by moment invariants," *IRE Transactions on Information Theory*, vol. 8, no. 2, pp. 179-187, 1962.

[3] M. K. Kalera, S. Srihari and A. Xu, "Offline signature verification and identification using distance statistics," *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 18, no. 7, pp. 1339-1360, 2004. (CEDAR database)

[4] L. G. Hafemann, R. Sabourin and L. S. Oliveira, "Offline handwritten signature verification: Literature review," *Proc. IPTA*, 2017.

[5] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed., Pearson, 2010.

[6] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018.

[7] S. G. Mallat, "A theory for multiresolution signal decomposition: the wavelet representation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 11, no. 7, pp. 674-693, 1989.

[8] J. P. Lewis, "Fast normalized cross-correlation," *Vision Interface*, 1995.

[9] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

[10] The MathWorks, *MATLAB Documentation: App Building and Image Processing*, R2026a.

APPENDIX A: PROJECT FILE LISTING

The complete project consists of 18 MATLAB files, about 2,200 lines in total, all written for this project.

Table 7: Project files and line counts

File	Lines	Purpose
signatureGUI.m	504	Graphical user interface
main_signature_verification.m	145	End-to-end evaluation and figures
tuneParameters.m	214	Grid search of weights and k
trainClassifier.m	183	Pair classifier and LOWO validation
registerPerson.m	72	Register an account
checkSignature.m	38	Verify against a registered account
listRegistered.m	35	List registered accounts
importCEDAR.m	58	Import the CEDAR database
src/preprocessSignature.m	237	Preprocessing with toolbox fallbacks
src/extractFeatures.m	243	137-D feature extraction
src/enrollUser.m	90	Enrolment and threshold
src/signatureScore.m	87	Distance, NCC, fusion
src/verifySignature.m	122	Decision and analysis figure
src/evaluateSystem.m	87	FAR, FRR, EER for one mode
src/looScores.m, classStats.m	33	Leave-one-out statistics
src/classifierScore.m	20	Classifier probability
src/makeSignatureDataset.m	112	Synthetic dataset for demos

APPENDIX B: HOW TO RUN AND REPRODUCE

Requirements: MATLAB R2020b or newer (for `uifigure` and `exportapp`), or GNU Octave for the command-line functions. No toolbox is required.

Listing B.1: Commands

```
>> cd E:\dsp
>> signatureGUI                     % interactive front end


% Bank workflow
>> registerPerson('ACC-1001', 'data/writer01')
>> R = checkSignature('ACC-1001', 'cheque_scan.png', true);
>> R.decision, R.score, R.threshold, R.confidence


% Full reproduction on CEDAR (about 60 minutes without toolboxes)
>> importCEDAR('path\to\signatures', 'data_train', 1:35)
>> importCEDAR('path\to\signatures', 'data', 36:55)
>> trainClassifier
>> tuneParameters
>> main_signature_verification
```

APPENDIX C: SAMPLE ACTIVITY LOG

The following lines are taken from `logs/signatureGUI.log`, recorded during the session used for the screenshots in Chapter 8.

Listing C.1: logs/signatureGUI.log

```
2026-10-26 20:41:35  INFO  GUI started. Log file: E:\dsp\logs\signatureGUI.log
2026-10-26 20:41:46  INFO  Enrolling "writer01" from 6 specimens in
C:\Users\Ayush\AppData\Local\Temp\writer01
2026-10-26 20:41:46  INFO  Trained classifier found, using hybrid DSP + ML scoring.
2026-10-26 20:41:46  INFO  Tuned parameters: weights [0.2 0.8], kThresh 1.50
2026-10-26 20:41:47  OK    Enrolled "writer01" [hybrid] in 1.3 s: 137 features,
threshold 0.607
2026-10-26 20:41:47  INFO  Leave-one-out genuine scores: 0.637 0.723 0.748 0.721
0.606 0.727
2026-10-26 20:41:50  OK    Loaded genuine_07.png: Otsu level 0.639, deskew -9.2 deg,
2000 ink pixels
2026-10-26 20:41:50  OK    genuine_07.png vs writer01: AUTHENTIC (score 0.638,
threshold 0.607, confidence 7.9%) in 0.49 s
2026-10-26 20:41:53  OK    Loaded forged_skilled_05.png: Otsu level 0.714, deskew -
2.0 deg, 2225 ink pixels
2026-10-26 20:41:53  ALERT forged_skilled_05.png vs writer01: FORGED (score 0.145,
threshold 0.607, confidence 76.1%) in 0.17 s
2026-10-26 20:41:59  INFO  Opened DSP analysis figure for forged_skilled_05.png.
```